Developer 2 Guide
EDITOR

pack v2.3.5 (final) | VS Code UI, per-hunk diff review, safe apply and undo, context finder, demo video

Forge-a-thon (TrueFoundry x Techfest IIT Bombay 2026-27). This guide replaces every earlier Developer 2 guide.

Every prompt in it is copied word for word from the pack's cards (`ai-guides/`). The pack is the source of truth.

Read this first (3 minutes)

You build everything the user **sees and decides**: the Ctrl+K inline edit, the agent panel, the per-hunk diff review, the safe Apply and Undo, and the **context finder** that picks which code the AI sees. Developer 1 builds everything that talks to TrueForge. You meet only at `src/types/contract.ts`. You do the thinking, steering and checking; your AI coding tool does the typing, **one small card at a time**. You work against `MockEngine` until Developer 1 says the real engine streams (`forge.useMocks=false`).

Your scoring jobs

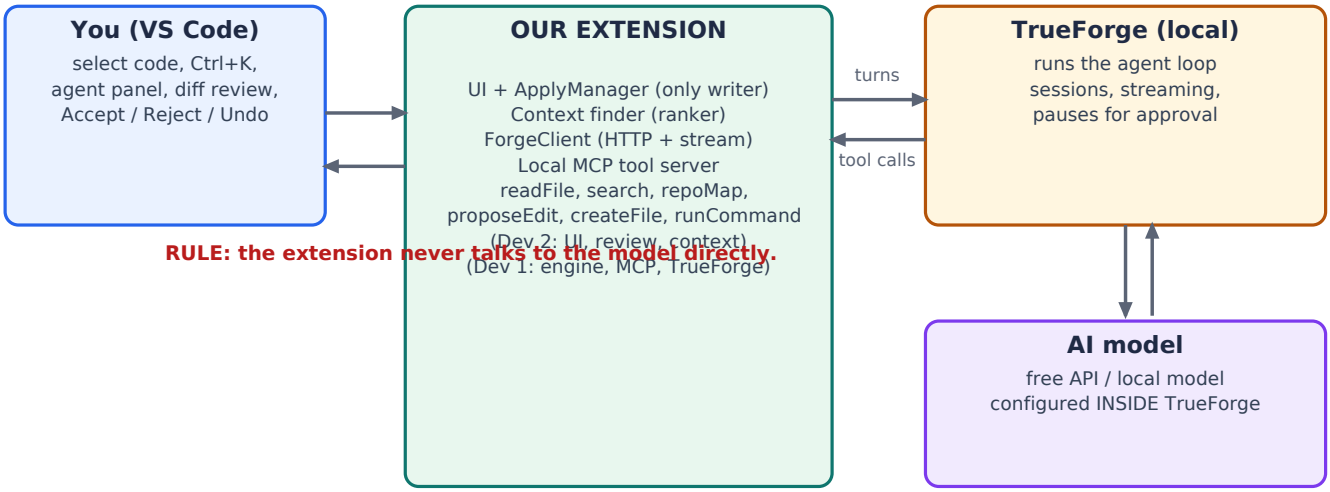
- **A1 inline edit (10)** and **B2 diff safety (10):** per-hunk accept/reject and a working Undo, shown on screen. This depends on CodeLens rendering inside the diff: spike it first (U-02).
- **B1 context assembly (15):** the biggest single item. Automatic retrieval with a size budget, strategy written down, and "Files chosen and why" visible in Task B.
- **A3 (10, UI side)** Stop is immediate and the editor never freezes. **C1 video (10)** and write-up sections **4** and **6**.

> **Status on Wed 11 Nov 2026 = Day 8 of the 15-day plan**
> Nothing in the pack proves the TrueForge spike has been run. Today: set up (Part 2), clone after Developer 1 merges S-01, help with the spike (S-02), and start **U-01 hunkText** (pure code, needs nothing from TrueForge) plus the **20-minute CodeLens spike** at the start of U-02. The re-baselined calendar is in Part 4. Submit by **Wed 18 Nov**; hard deadline Fri 20 Nov 23:59 IST.

Contents

Part	Title
1	Your half in plain words (VS Code concepts you need)
2	Setup today: tools, clone, worktree, running the extension
3	Free AI tools and models, and the rules for keys
4	Your re-baselined plan (Day 8 to Day 15)
5	The build: cards U-01 to U-13
6	Handshake with Developer 1 and the integration moments
7	The video and your write-up sections
8	Final checklists and live-call questions
9	When things go wrong

Part 1. Your half in plain words



Your side is the green box's **UI, ApplyManager and context finder**. The extension never calls a model. The engine (Developer 1) streams `EngineEvents` to you; you show them and send back one `reviewDone` per review batch.

What you own, and what you never touch

You own	Never touch (stop and ask Developer 1)
`src/extension.ts`, `src/ui/`, `src/review/`, `src/context/`, `media/`, `test/ui/`, `test/context/`, `test/e2e/`, `src/mocks/MockEngine.ts`, `package.json` **"contributes" only**, docs 04 and 05, write-up sections 4 and 6, the video	`src/engine/`, `src/mcp/`, `forge/`, `scripts/` (except `scripts/eval-context.ts`, agreed with Dev 1), `package.json` dependencies, the contract
Shared (never alone): `src/types/contract.ts`, `fixtures/`. Change requests: `docs/contract-change-requests.md`, joint PR.	**Docs owned by Dev 1:** 01-03, 06, 07 and write-up sections 1, 2, 3, 5, 7, 8

VS Code concepts you need (plain words)

Concept	What it is, and where you use it
Extension, activate, command	A TypeScript plugin. `src/extension.ts` runs when VS Code activates it and registers **commands** (like `forge.inlineEdit`) and **keybindings** (Ctrl+K). Declared in `package.json` under "contributes".
Extension Development Host (F5)	Press F5 in the extension's window: a second VS Code window opens with your extension loaded. Use the "Run Extension (mock)" or "Run Extension on Flask" launch configs.
Output channel	A log window in VS Code. The engine's state-machine log appears here: your first debugging tool.
`vscode.diff` + **TextDocumentContentProvider**	We show the proposal as a side-by-side diff. The right side is a **virtual document** (scheme `forge-proposed`) that we generate from the original text plus the accepted hunks. Nothing is written to disk.
CodeLens	The small clickable "Accept | Reject" text above a line. We place one pair above each hunk. **Unverified in a diff editor**, so U-02 starts with a 20-minute spike.
Hunk	One contiguous change. Lines are **1-based**; a pure insertion has `oldLines = 0` and goes BEFORE `oldStart`. Hunks are sorted and non-overlapping.
WorkspaceEdit	VS Code's way to change several files in one action. It is grouped but **not transactional** (a failure can leave some files changed), so ApplyManager verifies after applying and restores from a snapshot.
Stale file	The user edited the file after the agent proposed a change. We compare sha256 of the current text to the proposal's base hash; different means that file is refused (never merged).

Concept	What it is, and where you use it
Webview	A mini web page inside VS Code (our agent panel). Strict rules: a CSP with a nonce, `textContent` only, never `innerHTML` with model or repo text.
Workspace Trust	VS Code can mark a folder untrusted. The engine refuses `runCommand` then; you report `isWorkspaceTrusted()` through `UiHost`.

The review pipeline



Three safety rules that protect the user's files

1) **Only ApplyManager and UndoManager write** workspace files. 2) **Never** `saveAll`: save only the documents this apply modified, check each result (it would otherwise silently save the user's unrelated unsaved tabs). 3) **Never claim a rollback you did not verify**: before applying, persist the before-texts to a recovery file in `globalStorageUri`; after a restore, re-read and compare; if that fails report `restore_failed` with the recovery path and show a critical error.

If CodeLens does not work in the diff: the fallback ladder (docs/01 §7.1)

1. Comments API threads anchored at each hunk line with Accept/Reject commands (comments do render in diff editors).
2. Open the preview document in a NORMAL editor beside the diff, with line decorations and CodeLens.
3. Per-hunk buttons inside the agent panel. **Build option 3 in every case** (it is cheap and always works), so the demo never depends on one rendering trick.

How context is chosen (the 15-point item), in plain words

Flask is too big to paste to a model. So before the agent starts we build a small **seed**, and during the run the agent can search with strictly capped tools. Everything is counted in **one run budget**, and the panel shows a single meter plus a list of "files chosen and why".

Layer	What	Budget
1 @mentions	`@path` gives the file (up to 400 lines, else an outline); `@symbol` gives its body (up to 80 lines)	6,000 tokens total by default
2 Seed	Automatic retrieval before the agent starts, with reasons	3,000 tokens (a setting), including a tiny repo map
3 Agent tools	`repoMap`, `searchWorkspace`, `findSymbol`, `readFile` with hard caps	per tool
4 Run budget	One counter for seed + mentions + every tool result (scaled to the model's window)	30,000 by default; warning at 80%
5 Transparency	`reportContext` + seed reasons -> "Files chosen and why" and the meter	always visible

The ranking method (BM25F, no AI needed): split the request into words (snake_case and camelCase split, stopwords dropped); score each file on its path words (weight 3), symbol names (3), headings/docstrings (2) and body (1); then add structure signals: **test affinity**, **doc affinity**, **import neighbours**; reserve slots for a test and a doc when the request mentions them; limit to 3 files per directory in the top 8 (diversity); penalise huge and generated files. Each result carries a **reason** ("path: cli; symbols: ...; test of cli.py"). Token estimate is chars/4. **No project-specific word or filename may appear in `src/context`.**

Targets: recall@10 at least 0.8 overall and at least 0.7 on filename-free prompts; seed within budget 100% of the time; Flask index build under 5 s; search under 200 ms.

Part 2. Setup today

Tools (about 45 minutes)

Tool	Check	Notes
Node.js 22.14 or newer	`node -v`	If you have 20.x (for example 20.19.0), **upgrade now** (nodejs.org LTS or nvm-windows) and reopen the terminal.
Git, **VS Code**, **Python 3.10+**	`git --version`, `code --version`, `python --version`	Set `git config --global user.name` and `user.email`. Python is for the Flask demo and tests.
GitHub account	log in	Developer 1 adds you as a collaborator; accept the invite.
Techfest registration	Team ID saved	Registration closes **15 Nov**; confirm Developer 1 did it.
AI coding tool	Part 3	Pick one primary and one backup.

Clone, worktree, run

TERMINAL / CODE

```
git clone https://github.com/DEV1_NAME/forge-agent.git
cd forge-agent
git worktree add ../forge-dev2 -b dev2/u-01-hunktext main
cd ../forge-dev2
npm ci            # after S-01 is merged (it creates package.json)
npm run check     # typecheck + lint + unit tests + forbidden-pattern grep
# open the folder in VS Code, press F5 -> "Run Extension (mock)"
# in the new window: Ctrl+Shift+P -> "Forge: Show Demo Diff (mock)"
```

Until Developer 1 merges S-01 there is no `package.json`: start by reading `AGENTS.md`, `docs/00`, `docs/01`, `docs/04` and `src/types/contract.ts`, and prepare the Flask worktrees (below).

Get Flask ready (you will record the video)

TERMINAL / CODE

```
git clone https://github.com/pallets/flask.git
cd flask && git checkout 3.1.3 && git rev-parse --short HEAD   # starts with 22d92470
python -m venv .venv      # Windows: py -m venv .venv ; activate: .venv\Scripts\Activate.ps1
pip install -e . pytest
pytest tests/test_cli.py -q     # baseline: must pass BEFORE any agent run
# one worktree per demo segment: git worktree add ../flask-A 3.1.3 (then flask-B, flask-C, flask-
 stop)
```

PowerShell version of the Task A check

`$env:FLASK_DEBUG="off"; python -c "from flask.helpers import get_debug_flag; print(get_debug_flag())"` (the contest's `FLASK_DEBUG=off python -c ...` form is for bash). Record the Python version in `docs/rehearsal-log.md`.

Run every card the same way

1. Open your own worktree in its own VS Code window; start your AI tool from that folder in a **fresh** session.
2. Paste the **Session start** prompt (above). Wait for its 6-line summary.
3. Paste the card's prompt (the boxed text in this guide; it is word for word the pack's card, followed by the standard footer). Add the excerpts the card asks for (spike report, fixtures).
4. The tool answers with a **plan**. Check: only your folders? tests included? any invented TrueForge fact? Reply `go` or correct it.
5. It implements and runs `npm run check`. **You run the thing yourself.** "It compiles" is not "it works".
6. `git diff --stat`, revert anything outside your folders, commit with the card's message, push, open a PR. Your teammate reviews against "Done when".

PASTE FIRST IN EVERY FRESH AI SESSION

```
Read AGENTS.md, docs/00-PROJECT-BIBLE.md, docs/01-ARCHITECTURE.md and src/types/contract.ts.
I am Developer 2. Summarise in 6 lines: my owned paths, the hard rules, and the card I am about to
  give you.
Then wait. Do not edit anything yet.
```

The routine in commands

TERMINAL / CODE

```
git worktree add ../forge-devN -b devN/<card> main   # once per card (N = 1 or 2)
cd ../forge-devN && npm ci            # node_modules are per worktree
# ... run the card in your AI tool ...
git status && git diff --stat         # anything outside YOUR folders? -> git restore <file>
git add <only the files you meant to change>
git commit -m "<message from the card>"
git push -u origin devN/<card>        # then open a PR; teammate reviews against "Done when"
git fetch origin && git rebase origin/main   # after each merge
```

Hard rules your AI tool must obey (AGENTS.md)

No direct model-provider calls. Only ApplyManager/UndoManager modify workspace files. No shell strings. **No task-specific words** (`get_debug_flag`, `cli.py`, `flask config`) in `src/context`, the panel or tests of production code. Webview: `textContent` only and a strict CSP with a nonce. TypeScript strict, no `any` without a reason. Never edit the contract alone. No new dependencies (Developer 1 owns them). If a task needs a file you do not own: **STOP** and tell Developer 1.

Part 3. Free AI tools and models

There are **two different jobs** for AI. Do not mix them up. Facts below were read from public pages on 5-11 Nov 2026; free limits change often, so re-check before relying on one.

Job	What it is	Where you set it up
1. The tool that writes our code	A terminal or editor assistant that edits files for you. Any tool is allowed. Declare it in the write-up (it costs no points).	Your computer, your own worktree.
2. The model inside TrueForge	The "brain" the finished extension uses in the demo. Must be configured **inside TrueForge**. Two different providers: primary and fallback.	TrueForge UI -> model providers. Keys live only there.

Job 1: free tools that write the code

Tool	Free access (as found)	Catches
Freebuff (terminal agent). `npm install -g freebuff`, then run `freebuff` inside your worktree.	No subscription or key; ad-supported. Outside full-access regions about 5 h/day. Several free models.	May use prompts and code to personalise ads: keep its cloud/GitHub-repo connection OFF; some models say "may train on data"; availability depends on capacity.
Antigravity CLI (Google)	Replaced the free Gemini CLI on 18 Jun 2026; sign in with a Google account. No official free quota published; community reports a small starter quota.	The old Gemini CLI no longer serves free Google sign-ins. Test it on day one before relying on it.
GitHub Copilot (VS Code)	Free plan about 2,000 completions + 50 chat requests/month (third-party summary). Students can apply to GitHub Education for the Student plan.	Small monthly chat count; check GitHub's plans page.
Free web chat (Claude.ai, ChatGPT, Gemini)	Free accounts with message limits.	Copy-paste only, no file access. Great for explaining errors and reviewing a diff you paste.
Cline / Continue / Roo Code / opencode	Open source; bring your own key or a local model.	They have their own agent loops: fine as helpers, but never copy their loop into our extension (eligibility).
Claude Code	Needs a paid plan or API credit.	Not free. Good for E-06 and U-03 if one of you has it.

Job 2: free models to put inside TrueForge

Provider	Free tier (as found)	How	Catches
Google AI Studio (Gemini API key)	No card. Roughly 5-15 req/min and 20-1,500/day depending on model; Google cites 250/day for Flash on an unpaid key.	Catalog provider in TrueForge	HTTP 429 on per-minute caps; free-tier prompts may be used for training outside EU/UK/EEA: public Flask code only, never secrets.
Groq	No card. About 30 req/min, about 1,000/day; Llama-70B-class; 128K context.	OpenAI-compatible `https://api.groq.com/openai/v1`	Pick a model that supports tool calling.
OpenRouter	Free models about 20 req/min but only 50/day (about 1,000/day after a $10 top-up).	`https://openrouter.ai/api/v1`	Tiny daily cap: use as the fallback.
Cerebras	About 1M tokens/day, about 30 req/min.	OpenAI-compatible	Alternative.
Ollama (local)	Free; 7-8B models need about 16 GB RAM.	`http://localhost:11434/v1` (confirm in Ollama docs)	Weak at tool calls: use for the "weak model" drill only.

Rules for every AI tool and key

1) **Never paste a key, token or personal data into any AI tool**, a commit, a screenshot or the video. 2) Keys live only inside TrueForge (UI -> model providers). 3) If a key leaks: delete it at the provider first, then make a new one. 4) Primary and fallback must be **different providers**, otherwise Task C does not show a real switch. 5) Free tools have small windows and daily limits: one card per fresh session, commit after every card, keep a request tally in `docs/rehearsal-log.md`, and use `MockEngine`/fixtures instead of the live model whenever you can.

Handy prompts

When	Paste this
Plan first	Do not edit anything yet. Give me a plan: files to create or change, order, tests, and what could go wrong. Wait for my "go".
Tests first	Write the tests first and show me they fail for the right reason. Then implement until they pass. Do not change a test to make it pass.
Error	Here is the error: <paste exact text>. Explain the cause in 3 lines in simple words, propose the smallest fix, and do not refactor unrelated code.
Review before commit	Review my uncommitted diff. List: files outside my ownership, any hardcoded key/path/task word, any direct provider call, any workspace write outside ApplyManager/UndoManager, any shell string, any missing test. Be strict.
Learn	Explain <concept> to me like I am 12, then show a 10-line example from this project and name the file that uses it.
Stuck	Stop. You have tried the same approach several times. List 3 different hypotheses for the cause, the quickest experiment for each, and which you would try first.
Judge mode	Act as a strict Techfest judge scoring docs/00 section 11. For each criterion say which anchor our current code earns and the evidence, and list the cheapest improvement.
Live-call practice	Interview me as a Techfest judge about my half of the project. Ask one question at a time, grade my answer, and point out anything I got wrong.

EVENING EXPLAIN RITUAL (SAVE THE ANSWER IN DOCS/NOTES-DEVN.MD IN YOUR OWN WORDS)

```
Explain what you wrote today as if I must defend it to a judge: for each file, what it does, why
 this design, and what breaks if it is removed. Then ask me 5 questions to test my understanding.
```

Tip for your half

Most of your code is pure logic (`hunkText`, ranker, mentions, budget). Ask your AI tool for tests first, including property tests; weaker free models make fewer mistakes when each card is small and testable. Use `MockEngine` for all UI work so you never spend free-tier requests on layout changes.

Part 4. Your re-baselined plan (Day 8 to Day 15)

From `docs/06`, if the spike and the contract freeze are not done. Your column is in bold.

Day	Date	Dev 1	You (Dev 2)	Gate / check
8	Wed 11	registration, S-01, S-02 spike	**clone, help with the spike, U-01 hunkText, then the 20-minute CodeLens spike (start of U-02)**	G0 spike answers
9	Thu 12	S-03 freeze, E-01, E-02, E-03	**U-02, U-03, U-04**	G1 frozen
10	Fri 13	E-04, E-05, E-06	**U-05 Ctrl+K, U-08 panel**	**I-1 Task A end to end by night**
11	Sat 14	E-07, E-08, E-10	**U-06 ranker, U-07 provider, U-09 batch review**	**I-2 first Task B diff**
12	Sun 15	E-09 resilience	**polish the three tasks, U-12**	**G5 feature freeze (end of day)**; registration closes
13	Mon 16	rehearsals, README	**rehearse A, B, C; video rough cut; write-up 4 and 6 drafts**	all tests 3 of 3
14	Tue 17	final video + write-up PDF	**record the final video; fill scorecard timestamps**	
15	Wed 18	SUBMIT	**SUBMIT; open links logged-out**	**G6**

> **Cut lines for you**
> **U-10 (retrieval eval)** minimum if behind: Flask only, 12 cases, no second repo, an honest report. **U-11 (e2e)** minimum: automated diff-safety and cancel unit tests plus one manual run of each matrix item that appears on camera. Cut accessibility polish and the second local-model drill first. **Never cut:** per-hunk review, undo, clean cancel, the visible context list and meter, video quality, an honest scorecard.

Part 5. The build: cards U-01 to U-13

Paste each card after the Session-start prompt, one card per fresh session, one small PR per card. Work against `MockEngine` until Developer 1 says the real engine streams.

Your part in the spike (S-02) and the contract freeze (S-03)

Developer 1 leads the spike; you help. While Developer 1 runs the scripts, you (1) read `docs/02-TRUEFORGE-FACTS.md` so you know what is verified, (2) open TrueForge's own UI and check that an API-created session shows turns, tool calls, approvals and sub-agent threads (question Q16: the video must show it), (3) watch the raw event files in `fixtures/events/` so you understand what the engine will send you, and (4) do the 20-minute CodeLens test. Then you and Developer 1 freeze the contract together.

Contract freeze (both) — Day 2 morning, Gate G1

What this is, in plain words. Both of you read what the spike taught, agree on any contract changes and freeze `src/types/contract.ts`. After the freeze neither of you edits it alone.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Read docs/spike-report.md and src/types/contract.ts. List every place the contract disagrees with
 real TrueForge behaviour or fixtures
(event naming, approval payload, error kinds, review batch shape). Propose the minimal diff to the
  contract. Do not edit the file yet; output the diff and a one-paragraph rationale.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when:

Check it yourself: The PR changes only the contract and fixtures; `CONTRACT_VERSION` is `2.1.0` (or 2.2.0 if the spike changed the shape). Check the pieces you depend on: `FileProposal.callIds`, `ReviewResult` (including

`restore_failed` and `recoveryPath`), `UiHost.isWorkspaceTrusted()`, and the single `budget` event.

Watch out: Do not approve a contract you do not understand. Ask the AI to explain each type in plain words first.

hunkText — rebuild text from hunks (Day 1)

What this is, in plain words. `hunkText` turns a proposal into text: with all accepted hunks it equals the proposed file, with all rejected it equals the original, and in between it shows exactly the accepted ones. It is pure code (no VS Code), so it is easy to test thoroughly. Everything in review and apply is built on it.

You may edit only: `src/review/hunkText.ts`, `test/ui/hunkText.test.ts`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Create a pure, vscode-free buildText(p: FileProposal, mode: 'preview'|'final') → {text, hunkLine}.
  Contract rules: 1-based lines; oldStart/oldLines = original lines replaced; pure insertion has
 oldLines 0 and goes BEFORE oldStart; hunks sorted and non-overlapping.
preview: pending and accepted hunks applied, rejected shown as original; final: ONLY accepted
 hunks applied. hunkLine[hunkId] = 0-based output line where that hunk starts (for CodeLens).
Preserve line endings per line: split into lines KEEPING each original terminator and rebuild from
  those, so mixed CRLF/LF files round-trip byte-identically (new lines use the EOL of their
 neighbouring line), handle no trailing newline, empty file, first/last line hunks.
Tests: all accepted == proposedContent; all rejected == originalContent; mixed; pure
 insert/delete; start/end of file; CRLF; MIXED CRLF/LF (byte-identical for untouched lines); no
 trailing newline; plus a property test (200 random hunk sets generated from random before/after
 texts via the 'diff' package; for every accept/reject combination final text equals the expected
 manual construction).

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: tests green incl. property test.

Check it yourself: All tests green, including the property test (200 random hunk sets) and a **mixed CRLF/LF** file: untouched lines stay byte-identical.

Watch out: Lines are 1-based; pure insertion goes BEFORE `oldStart`. Split lines KEEPING each terminator and rebuild from them; do not "detect one EOL style".

Commit message: `feat(review): hunkText with property tests`

DiffReviewer + per-hunk CodeLens (Day 2)

What this is, in plain words. The diff reviewer opens a side-by-side diff of original vs proposed and puts **Accept | Reject** CodeLenses above each hunk. The first 20 minutes are a throwaway spike to see whether CodeLens renders in a diff editor at all (20 of the 100 points depend on it).

You may edit only: `src/review/DiffReviewer.ts`, `HunkCodeLens.ts`, `src/extension.ts`, contributes.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
FIRST (20 minutes, before any other code; docs/01 §7.1): in a throwaway command open vscode.diff
 on two virtual documents and register a CodeLensProvider for the modified side's scheme; write
 docs/spike-codelens.md with a screenshot answering: do lenses render in the diff editor? If YES
 continue as written. If NO implement the fallback ladder: (1) Comments API threads at each hunk
 line with Accept/Reject commands, (2) the preview document in a NORMAL editor beside the diff
 with decorations + CodeLens, (3) per-hunk buttons in the agent panel (built in every case, see
 U-08/U-09).
Then implement DiffReviewer: TextDocumentContentProvider scheme 'forge-proposed' rendering
 buildText(p,'preview').text; open vscode.diff(original uri, proposed uri); keep an ordered list
 of proposals (source files, then tests, then docs by path heuristics: contains 'test' → tests;
 .md/.rst/.txt → docs); setHunkStatus fires onDidChange for that URI and refreshes CodeLens.
HunkCodeLens: for scheme 'forge-proposed' return two lenses per hunk at buildText(...).hunkLine:
 "$(check) Accept" and "$(x) Reject" (show state), commands
 forge.acceptHunk/forge.rejectHunk(proposalId, hunkId); also
 forge.acceptAllHunks/forge.rejectAllHunks for the active diff. Hunks start 'pending'.
Never write files here. Test with "Forge: Show Demo Diff (mock)" (2 hunks): rejecting hunk 2
 reverts it on the right side; lens positions stay correct after every change.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: per-hunk review works on the demo diff (A1 top anchor territory).

Check it yourself: Write `docs/spike-codelens.md` with a screenshot. Then on "Forge: Show Demo Diff (mock)" (2 hunks): rejecting hunk 2 reverts it on the right; lens positions stay correct after every change.

Watch out: If CodeLens does not render, follow the fallback ladder (Part 1) in order, and build the panel's per-hunk buttons anyway. This card never writes files.

Commit message: `feat(review): per-hunk accept/reject with CodeLens`

ApplyManager + UndoManager (Day 3) — B2 core

What this is, in plain words. The **only writer** (with UndoManager). It re-checks each file is unchanged since the proposal, builds ONE WorkspaceEdit across files, verifies the result, saves only the touched documents, and records before/after for Undo. If anything fails it restores and tells the truth.

Read first: docs/01 §7, §8.
You may edit only: `src/review/ApplyManager.ts`, `UndoManager.ts`, `src/extension.ts`, `test/ui/apply*.test.ts` (logic split into vscode-free helpers).

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
ApplyManager is the ONLY writer (with UndoManager). apply(proposals): for each proposal with ≥1
 accepted hunk: re-snapshot the document (open document text if open else disk); if sha256(text)
 != proposal.base.contentHash → mark failure {kind:'stale'} and DO NOT write that file; never
 merge. Compute finalText = buildText(p,'final').text. Build ONE vscode.WorkspaceEdit across files
  (replace whole range; createFile + insert for new files). Record before-text (null for new
 files) and after-hash BEFORE applying. After applyEdit returns, re-read each document and verify
 text === finalText; mismatch or false → restore from snapshot, re-read and VERIFY the restore,
 and report failure {kind:'apply_failed'} (or {kind:'restore_failed', recoveryPath} + critical
 notification if restoration itself fails; before applying, persist before-texts to a recovery
 file in globalStorageUri). Then save ONLY the documents this apply modified (for each: await
 document.save(), check the returned boolean, verify disk equals text). NEVER call saveAll: it
 would save unrelated dirty tabs. Return ApplyRecord and a ReviewResult-compatible list.
Note in code comments: WorkspaceEdit is grouped but NOT transactional; our verify+restore is the
 safety net.
UndoManager: stack of ApplyRecords; undoLast restores before-text or deletes created files;
 refuses with an explanation (and confirm option) if a file's current hash != afterHash (edited
 since); shows "Forge: undone N files". Commands: forge.applyAccepted, forge.undoLastApply.
Pure helpers (hashing, planning, verification) must be vscode-free with unit tests; add extension-
 host tests later (U-11). Pending hunks are NOT applied.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: an unrelated dirty tab is still dirty/unsaved after Apply; injected restore failure produces restore_failed + recovery file; demo diff: accept 1, reject 2, Apply → only hunk 1 on disk; Undo restores; editing the file during review makes Apply refuse that file.

Check it yourself: An unrelated dirty tab is still unsaved after Apply; an injected restore failure gives `restore_failed` plus a recovery file; accept 1 and reject 2 then Apply writes only hunk 1; Undo restores; editing the file during review makes Apply refuse that file.

Watch out: Never `saveAll`. Never merge into a stale file. Pending hunks are NOT applied. Keep the logic in vscode-free helpers so it is unit-testable.

Commit message: `feat(review): safe apply with stale detection and snapshot undo`

UiHost + EventRouter + StatusBar (Day 2–4)

What this is, in plain words. `UiHost` is what you give the engine: read a file, take a snapshot, run a command (argv only, scrubbed environment, locked to the workspace), ask for confirmation, report Workspace Trust. `EventRouter` is the one place that turns engine events into UI: output, timeline, files list, meter, diffs, toasts.

You may edit only: `src/ui/index.ts`, `EventRouter.ts`, `StatusBar.ts`, `src/extension.ts`, `src/review/*` (wiring only).

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Implement createUiHost(): snapshot(path) (open document text/version/isDirty/sha256, else disk),
 readFile(path, range) (workspace-relative, forward slashes, bounds), workspaceRoot, context
 (U-07), confirmCommand (non-modal notification with buttons Allow once / Allow for this run /
 Deny; resolves false on dismiss), runCommand(req, signal): child_process.execFile with
 shell:false, cwd locked to workspaceRoot (reject any other cwd), env scrubbed per docs/03 (never
 forward *_KEY/*_TOKEN/*_SECRET/TRUEFORGE_*), timeout, maxBuffer, kill the process TREE on
 abort/timeout (taskkill /T on Windows, process group on POSIX), tail-trim output to
 maxOutputChars, return CommandResult.
EventRouter: single place mapping EngineEvent → UI: token → output channel + panel;
 tool.call/result → timeline; context.report → files list; budget → meter; subagent → timeline
 marker; proposal.created → DiffReviewer.add; review.required → open first diff, show "N files to
```

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT) (CONTINUED)

```
 review" in status bar; proposal.discarded → close diff + remove; retry → "retrying k/3" status;
 failover → warning toast + model name refresh; run.failed → non-modal error toast with friendly
 message (never stack traces); run.completed → "Forge: no changes proposed" if none; run.cancelled
  → clear state. Always return to idle in finally. Never await a run inside a command handler
 (void router.consume(...)).

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: MockEngine scenarios `mock:error`, `mock:failover`, `mock:cancel` behave: toast/spinner/diffs cleared; editor stays responsive.

Check it yourself: The MockEngine scenarios `mock:error`, `mock:failover` and `mock:cancel` behave: the toast or spinner shows, diffs are cleared, the editor stays responsive.

Watch out: `confirmCommand` resolves false if the user dismisses it. `cwd` must be the workspace root: reject anything else.

Commit message: `feat(ui): UiHost, event router and status bar`

Ctrl+K inline edit (Day 4) — Task A

What this is, in plain words. Task A: press Ctrl+K on a selection, type the instruction, and a diff appears. Friendly messages for no editor, empty selection, untitled file or a file outside the workspace. The editor must not freeze while tokens stream.

You may edit only: `src/ui/InlineEdit.ts`, `src/extension.ts`, contributes (keybinding).

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Command forge.inlineEdit bound to ctrl+k / cmd+k only when editorHasSelection && editorTextFocus
 (and a second binding ctrl+alt+k always). Capture workspace-relative forward-slash path, selected
  text, 1-based range. Friendly messages for: no editor, empty selection, untitled file, file
 outside workspace. showInputBox for the instruction (Escape = nothing). Call
 engine.runInline(...) via router.consume without awaiting; status bar shows running + Stop.
When the run completes with no proposal: "Forge: no changes proposed". Do not freeze the editor:
 test typing in another tab while tokens stream (MockEngine).

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: I-1 Task A on Flask with real engine, exact wording → diff with only the target function changed.

Check it yourself: Integration check **I-1**: Task A on Flask with the real engine and the exact wording gives a diff with only the target function changed.

Watch out: Bind Ctrl+K only when there is a selection (and a second binding, Ctrl+Alt+K, always). Test typing in another tab while tokens stream. Use the Flask worktree `flask-A`.

Commit message: `feat(ui): robust Ctrl+K inline edit`

Indexer + ranker + budget (Day 5) — B1 core

What this is, in plain words. The context finder's brain: index the workspace (chunked so the editor never blocks), then rank files for a request with BM25F plus test, doc and import signals, with a reason for every result. This is the intellectual core of B1.

Read first: docs/04 whole.
You may edit only: `src/context/indexer.ts`, `ranker.ts`, `budget.ts`, `test/context/*`, contributes (settings `forge.context.*`).

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Implement per docs/04: WorkspaceIndex (chunked async build yielding every ~100 files; findFiles
 with excludes; mtime+size cache; incremental update on save/watch; Python/TS/JS symbol regexes;
 md/rst headings; path words; import edges one hop via regex).
rank(prompt, index, cfg: RankerConfig) → [{path, score, reason}] using BM25F over fields (path,
 symbols, headings/docstring first lines, body) with weights from settings (defaults 3/3/2/1, k1
 1.2, b 0.75), intent-word categories (test/doc/config/cli by CATEGORY only), test affinity, doc
 affinity, import neighbours, slot reservation for tests/docs when asked, MMR diversity (≤3 per
 directory in top 8), noise penalties. Reasons must list matched fields/terms/signals.
budget.ts: estimateTokens(chars/4) and fitToBudget.
FORBIDDEN: any project-specific word or file name in src/context. Tests use small synthetic
 indexes AND fixture snapshots; add tests for identifier splitting, stopwords, intent slots,
```

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT) (CONTINUED)

```
 diversity, penalties, determinism.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: unit tests green; top-10 for a filename-free prompt looks sensible on Flask; build < 5 s.

Check it yourself: Unit tests green; the top 10 for a filename-free prompt looks sensible on Flask; the index builds in under 5 seconds.

Watch out: No project-specific word or filename in `src/context` (a judge will check). Weights are settings, not constants. Results must be deterministic.

Commit message: `feat(context): BM25F ranker with structure signals`

ContextProviderImpl + mentions (Day 6)

What this is, in plain words. The functions the engine's tools call: search, find symbol, repo map, resolve `@mentions`. All fast (under 200 ms on Flask) and never throwing.

You may edit only: `src/context/ContextProviderImpl.ts`, `repoMap.ts`, `mentions.ts`, tests.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Implement all ContextProvider methods using the index/ranker; never throw (return empty results);
 each <200 ms on Flask:
search({query, glob, maxResults}) line scan (case-insensitive; /regex/ if slashed), best-ranked
 files first, previews ≤160 chars, return omitted count; findSymbol (exact → prefix → contains,
 with kind); repoMap({dir, depth}) directories first, noise excluded, ≤1500 tokens with truncated
 flag; resolveMention('@path' | '@symbol') (file ≤400 lines else outline; symbol body ≤80 lines;
 null if missing; path must stay inside workspace); seed(prompt, budget) → ContextItems with
 range, reason, tokens, source 'auto', excluding files already mentioned; includes one tiny repo
 map ≤500 tokens within the budget.
parseMentions(text) pure + tests.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: Integration moment #2: engine tools return real, fast, capped results.

Check it yourself: Integration moment **#2**: with the real engine, the agent's tools return real, capped results quickly.

Watch out: A path must stay inside the workspace. A missing mention returns null, not an exception.

Commit message: `feat(context): context provider and mentions`

Agent panel webview (Day 6–7)

What this is, in plain words. The agent panel: streaming messages, a tool timeline, "Files chosen and why" with source badges, ONE budget meter, the list of proposed changes with a Review button and per-hunk buttons, and a Stop button. It is the screen the judges watch for B1.

You may edit only: `src/ui/AgentPanel/*`, `media/panel.js|css`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Build the agent side panel (WebviewViewProvider): CSP default-src 'none' with nonce,
 localResourceRoots=media only, plain JS, VS Code theme variables, textContent ONLY (never
 innerHTML).
Layout: (1) messages with incremental streaming (append to current node; autoscroll only if at
 bottom), (2) tool timeline rows (spinner→check/cross, short args, subagent markers,
 retry/failover notes), (3) "Files chosen and why" from seed + context.report with source badges
 (mention/auto/agent), (4) ONE budget meter: total run budget used/limit from budget events (the
 seed, mentions and tool results are all charged to it), with a breakdown on expand, (5) Proposed
 changes list (file, #hunks, rationale, Review button, status, and per-hunk Accept/Reject buttons
 for every hunk, always, independent of CodeLens), buttons Accept all / Reject all / Apply
 accepted / Undo last, (6) input with @mention autocomplete (up to 8 suggestions from the index)
 and chips, Send, Stop, active model name.
Keyboard accessible, readable at 150% zoom, big fonts for video. Panel must handle 500 timeline
 rows without lag.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: with MockEngine the panel tells the whole Task B story; judges can understand it by looking.

Check it yourself: With MockEngine the panel tells the whole Task B story and a judge can understand it by looking. Keyboard accessible, readable at 150% zoom and big enough for the video; 500 timeline rows do not lag.

Watch out: `textContent` ONLY; CSP `default-src 'none'` with a nonce; `localResourceRoots` = `media` only.

Commit message: `feat(ui): agent panel with timeline, context list and budget meter`

Multi-file batch review → reviewDone (Day 7)

What this is, in plain words. When the engine says a batch needs review, you open the first diff, list the others, and let "Apply accepted" write the whole batch once (one WorkspaceEdit, one Undo record). Then you **always** call `reviewDone` so the agent can continue.

You may edit only: `src/review/*`, `src/ui/AgentPanel/*`, `src/ui/EventRouter.ts`.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
On review.required{batchId, proposalIds}: open the first proposal's diff only; list others in the
 panel; status bar "Forge: N files to review". "Apply accepted" runs ApplyManager once for the
 whole batch (one WorkspaceEdit, one undo record), then calls engine.reviewDone({batchId,
 proposals:[{proposalId, decisions, applied, failure?}], applyId}) — ALWAYS call it, even if
 everything was rejected (applied:false) or stale, so the agent can continue. "Reject all" also
 calls reviewDone. Closing diff tabs does not decide anything.
If the run is cancelled/discarded while reviewing: close diffs, clear list, do not call
 reviewDone.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: I-2: 3-file batch applies in one action; one Undo reverts all; agent continues and runs tests.

Check it yourself: Integration check **I-2**: a 3-file batch applies in one action; one Undo reverts all; the agent continues and runs the tests.

Watch out: Call `reviewDone` even when everything was rejected or stale. If the run is cancelled or discarded while reviewing, close the diffs and do NOT call it. Identical calls arrive as ONE proposal (the engine maps ids), so you never see duplicates.

Commit message: `feat(review): batch review and reviewDone handshake`

Retrieval eval harness + second repo (Day 8–10) — evidence for B1

What this is, in plain words. STRETCH. Evidence for B1: a set of test prompts with the files that should be found, run through your real ranker, reporting recall@5/@10 and MRR on Flask and (if time) one second repo.

You may edit only: `scripts/eval-context.ts` (agreed with Dev 1), `fixtures/eval/*`, `docs/eval-report.md`, `docs/04` numbers.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Create fixtures/eval/<repo>/cases.jsonl for Flask 3.1.3 and ONE real second repo of a different
 shape (pin the commit in fixtures/eval/README.md): ≥12 cases each of kinds feature-in-words, bug-
 in-words, ambiguous, misleading-keyword, tests+docs-required, unconventional-naming, docs-only;
 ≥8 per repo with NO filenames or symbol names. expectedFiles are chosen by reading the repo, NOT
 by running our ranker.
npm run eval:context loads the real WorkspaceIndex + ranker (extract any vscode-dependent part
 behind an interface), prints recall@5, recall@10, MRR by kind and overall, seed tokens used vs
 budget, index time, and per-case misses. Split cases odd/even: tune weights only on odd; report
 even as the headline.
Write docs/eval-report.md: tables, weights used, 5 failure analyses, what we'd improve. Do not
 special-case any case.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: targets in docs/04 met or honestly reported.

Check it yourself: The targets in docs/04 are met, or honestly reported with failure analyses.

Watch out: Choose the expected files by reading the repo, **not** by running your ranker. Tune on the odd cases only and report the even cases. Never special-case a prompt.

Commit message: `test(context): retrieval eval on two repos with held-out split`

Extension-host e2e + cancel/failure UX (Day 9–11)

What this is, in plain words. STRETCH in part. Tests that run inside real VS Code: the diff-safety matrix, the cancel matrix on the UI side, event-loop lag while streaming, and the webview injection test.

You may edit only: `test/e2e/*`, `src/ui/*`, `src/review/*`.

> **PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)**

```
Add @vscode/test-electron tests (MockEngine + real ApplyManager/DiffReviewer on a temp workspace):
  diff-safety matrix in docs/05 (multi-hunk, multi-file, external edit during review → refused,
 unsaved base edit at proposal time → allowed, partial accept + undo, new file + undo, injected
 apply failure → restored, save failure, cancel while reviewing → nothing written); cancel matrix
 UI side; event-loop lag measurement (p95 < 50 ms while MockEngine streams 5 000 tokens); webview
 CSP check.
Fix every bug found. Verify typing/scrolling in another editor during streaming. Add the webview
 injection test from docs/05 (model/repo text such as `<img src=x onerror=alert(1)>`, `</script>`
 and a fake `vscode://` link must render as inert text; assert the panel DOM uses textContent only
  and the CSP blocks inline script).

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when: all matrices pass; lag target met.

Check it yourself: Automated diff-safety and cancel tests pass; event-loop lag p95 is under 50 ms while mock-streaming 5,000 tokens; a malicious string in model text renders inert.

Watch out: Minimum if behind: unit-level safety tests plus one manual run of each matrix item that appears on camera.

Commit message: `test(e2e): diff safety, cancel and responsiveness`

Polish for the camera (Day 10–11)

What this is, in plain words. Polish only what the video shows: clear notifications (no stack traces), consistent labels, big fonts, a "Switch Model" picker, status-bar states, keyboard shortcuts, and a visible "Undo last apply".

You may edit only: `src/ui/*`, `src/review/*`, `media/*`.

> **PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)**

```
Polish only what the video shows: clear notifications (no stack traces), consistent labels, big
 readable fonts in the panel, "Forge: Switch Model" QuickPick (profiles primary/fallback via
 engine.switchModel), status bar states (idle/running/retrying/reviewing N files/model), multi-
 file review ordering, keyboard shortcuts for accept/reject hunk, 'Undo last apply' visible after
 apply. No new features after G5.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when:

Check it yourself: Watch your own recording: every label is readable and no raw error text appears.

Watch out: No new features after the feature freeze (G5).

Commit message: `feat(ui): demo polish and model switch`

Video + write-up §4 and §6 (Day 12–15)

What this is, in plain words. Your two write-up sections and the video work. The scorecard lists one row per criterion with the video timestamp and a one-line claim, and you may only claim what the video shows.

You may edit only: `docs/writeup/04*.md`, `docs/writeup/06*.md`, `docs/rehearsal-log.md`.

> **PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)**

```
Draft write-up §4 (layers/budgets table, ranking method, eval numbers from docs/eval-report.md,
 one honest failure) and §6 (scorecard table; leave timestamps as __:__ until the final video
 exists; only claims the video shows). Then, after recording, fill timestamps and verify each by
 listing the shot at that time.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when:

Check it yourself: Every timestamp in the scorecard is verified by looking at the shot at that time.

Watch out: Rehearse A, B, C three times each from a clean worktree before recording.

Commit message: `docs: context section and scorecard`
Human work: rehearse A→B→C 3× (docs/05), record per docs/07 script, add on-screen captions (Task C: "fallback diff shown — deliberately not applied"), upload unlisted, verify the link logged-out.

Part 6. Handshake with Developer 1 and the integration moments

You consume `EngineApi` (`runInline`, `runAgent`, `cancel`, `reviewDone`, `switchModel`) and the `EngineEvent` stream, and you provide the `UiHost`. Never edit the contract alone.

Moment	When	What must work
#1 tokens stream	after E-03	Set `forge.useMocks=false`; see real tokens in the Output channel and panel.
#2 real tools	after E-04 + U-07	Engine tools return real, fast, capped results from your context provider.
I-1 Task A	Day 10	Exact Task A wording on worktree `flask-A`: diff with only the target function; Apply; check prints `False`; Undo works.
I-2 Task B	Day 11	Three runs on `flask-B`; record files touched and success; panel shows the files list and meter.
I-3 Task C	Day 12	On `flask-C`: wrong key on primary; fallback toast; diff appears; `git status` empty.
I-4 fresh clone	Days 13-14	A second machine follows only the README.

Integration checks (both, at the gates) — see docs/06 table

What this is, in plain words. A rehearsal log and verify scripts that only **check**. Use it at every integration moment.

PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)

```
Create docs/rehearsal-log.md with a table for AT-A, AT-B, AT-C × 3 runs: date, model, success
 (Y/N), files touched, seconds, failure notes.
Add scripts/verify-task-a.sh, verify-task-b.sh, verify-task-c.sh (+ .ps1 equivalents) per docs/05
 acceptance tests. Scripts only verify; they never edit files.

Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
 End with: what changed / how I verify by hand / what could go wrong.
```

Done when:

Check it yourself: Three runs of each acceptance test are logged.

Watch out: Keep your free-tier request tally.

Part 7. The video and your write-up sections

Demo video (maximum 4:00)

Time	Shot	Proves
0:00-0:12	Caption; TrueForge terminal + VS Code on Flask 3.1.3; `git status` clean.	C1
0:12-1:05	**Task A** (worktree A): select `get_debug_flag`, Ctrl+K, exact instruction, **no cut** until the diff appears, accept hunks one by one, Apply, run the check showing `False`, Undo then redo.	A1, B2
1:05-2:45	**Task B** (worktree B): exact instruction with no mentions; tool timeline, "Files chosen and why", budget meter; **show TrueForge's own session view for about 10 seconds** (turns, tool calls, approval, explorer thread); multi-file review; Apply; agent runs pytest (confirm prompt); run `pytest tests/test_cli.py -q`; run `flask config` showing `"***"`.	A2, B1, B4, A3
2:45-3:35	**Task C** (worktree C): break the primary key (an invalid key is faster than an unreachable endpoint), repeat Task A, show the retry or fallback toast and the fallback's diff, then `git status` empty. Say on screen that the fallback diff is shown and **not applied**.	B3
3:35-3:55	**Stop** mid-stream: cancelled, `git status` clean, editor still responsive.	A3
3:55-4:00	Closing caption.	

- Do not cut between giving an instruction and the diff appearing (you may cut model waiting time elsewhere). 1080p, big font, hide notifications and **any key or personal data**.
- Free recorders: OBS Studio, or Windows Game Bar (Win+G). Upload as YouTube **unlisted** or Google Drive "anyone with the link can view" and test it in a private window.
- Note timestamps while recording for the scorecard.

Your write-up sections

#	Section	What to put
4	How context is selected	The layers and budgets table, the ranking method in plain words, the recall@k numbers from `docs/eval-report.md` (or "Flask only, 12 cases" if you cut the second repo), and **one honest failure case**.
6	Scorecard	One row per criterion (A1, A2, A3, B1, B2, B3, B4, C1, C2): video timestamp and a one-line claim. **Only claim what the video shows.** Leave __:__ until the final video exists, then fill and verify every timestamp.

Use the "Write-up helper" idea: ask your AI tool for a draft from `docs/04`, then verify every number yourself.

Part 8. Final checklists and live-call questions

Release (Day 14–15)

What this is, in plain words. A strict PASS/FAIL report on the final checklist with real command output as evidence. The AI only reports; **you** fix.

> **PASTE INTO YOUR AI CODING TOOL (AFTER THE SESSION-START PROMPT)**
>
> ```
> Run the final checklist in docs/07-SUBMISSION-KIT.md. For each item output PASS/FAIL with command
>  output as evidence (secret scan, forbidden patterns, clean-clone build in a temp dir, README step
>   dry-run, license present, setup.example.json and models.example.json have no keys, write-up PDF
>  and video links are public, agent-instructions identical to forge/agent.spec.json's
>  instructions).
> Do not fix anything; just report.
>
> Plan first and wait for "go". Touch only the listed paths. Run npm run check and show the output.
>  End with: what changed / how I verify by hand / what could go wrong.
> ```

Done when:

Check it yourself: Every item is PASS.

Watch out: Developer 1 runs it; help fix the items in your folders.

Your definition of done

- ■ Per-hunk review and Undo work on screen; an unrelated dirty tab stays unsaved after Apply; stale files are refused.
- ■ CodeLens spike done (`docs/spike-codelens.md`) and the panel has per-hunk buttons.
- ■ Ctrl+K works on the exact Task A wording; the agent panel shows the files list and ONE meter during Task B; Stop is immediate.
- ■ No project-specific word in `src/context`; webview uses `textContent` only with a nonce CSP.
- ■ Retrieval report committed (or honestly reduced); Tasks A, B, C succeed 3 of 3 from a fresh clone; cancel and diff-safety checks pass.
- ■ Video 4:00 or less, A -> B -> C in order, TrueForge session shown in B, link public (open it logged-out).
- ■ Write-up sections 4 and 6 written; scorecard timestamps verified; three links submitted by Wed 18 Nov.

Live-call questions for you (10 minutes; Task A may be repeated with the organisers' model)

1. Walk through what the user sees from Ctrl+K to the applied diff.
2. How do you know a file was not edited during review? What happens if it was?
3. What happens if WorkspaceEdit fails halfway? How is that recovered and verified?
4. Why never `saveAll`?
5. Explain your ranking method and how you know it works. Show the numbers.
6. What is in the budget and what is left out? Where is it visible?
7. How does the editor stay responsive while streaming?
8. What did you do about CodeLens in a diff editor?
9. How do you prevent a malicious string in a repo file from affecting the panel?
10. What would you do with another month?

Practise with the "Live-call practice" prompt in Part 3.

Part 9. When things go wrong

Symptom	Likely cause	What to do
No `package.json`, `npm ci` fails	S-01 not merged yet	Pull `main` after Developer 1 merges; meanwhile read the docs and prepare Flask.
CodeLens does not appear in the diff	Not supported on that side of `vscode.diff`	Run the fallback ladder in order; build the panel buttons anyway.
Lens positions drift after a reject	`hunkLine` computed from the wrong text	Recompute from `buildText(p,'preview')` after every status change; add a test.
Apply refuses a file	Stale hash (edited during review)	Working as designed: re-run the request or re-open and re-review.
An unrelated tab got saved	Used `saveAll`	Save only the modified documents; add the "unrelated dirty tab" test.
The editor freezes while indexing or streaming	Work on the extension host thread	Chunk the index (yield about every 100 files); batch UI updates; measure event-loop lag.
Webview shows raw HTML from a file	Used `innerHTML`	Use `textContent`; add the injection test.
Free AI tool stops for the day	Quota used up	Switch to your backup tool; keep cards small; resume tomorrow.
AI edited files outside your folders	Vague request	`git restore` them; re-run the card and say "stay in my folders".
Ctrl+K does nothing	Keybinding conflict or no selection	Use Ctrl+Alt+K; check the when clause; show a friendly message.
Works on your laptop but not a fresh clone	Missing file or global install	Follow the README in a temp folder; tell Developer 1.
A key was pasted somewhere	Human error	Revoke it at the provider now; create a new one; tell Developer 1 to run the secret scan.